

Introduction to Parsing

What does the parser do?

Input: sequence of tokens from lexer

Output: Parse tree of the program.

- Cool

if x = y then 1 else 2 fi

- Parser input

IF ID = ID THEN INT ELSE INT FI

- Parser output



Phase	Input	Output
Lexer	<u>String of characters</u>	<u>String of tokens</u>
<u>Parser</u>	<u>String of tokens</u>	<u>Parse tree</u> (may be implicit)

Context-Free Grammars

Not all strings of tokens are programs
Parser must distinguish between valid and invalid strings of tokens.

We need:

- A language for describing valid strings of tokens.
- A method for distinguishing valid from invalid strings of tokens.

Programming languages have recursive structure.

An EXPR is

if EXPR then EXPR else EXPR fi
while EXPR loop EXPR pool
...

Context-free Grammars are a natural notation for this recursive structure.

A CFG consists of

- ① A set of terminals T
- ② A set of non-terminals N
- ③ A start symbol S ($S \in N$)
- ④ A set of productions

$$X \rightarrow Y_1 \cdots Y_n$$

$$X \in N$$

$$Y_i \in N \cup T \cup \{\epsilon\}$$

1. Begin with a string with only the start symbol S .

2. Replace any non-terminal X in the string by the right-hand side of some production $X \rightarrow Y_1 \cdots Y_n$

3. Repeat (2) until there are no non-terminals

Let G be a context-free grammar with start symbol S . Then the language $L(G)$ of G is:

$$\{\alpha_1 \dots \alpha_n \mid (\forall i: \alpha_i \in T) \wedge (S \xRightarrow{*} \alpha_1 \dots \alpha_n)\}$$

Terminals are so-called because there are no rules for replacing them.

Once generated, terminals are permanent.

Terminals ought to be tokens of the language.

The idea of a CFG is a big step. But:

- ① Membership in a language is "yes" or "no", also need parse tree of the input.
- ② Must handle errors gracefully.
- ③ Need an implementation of CFGs (e.g., bison)

Form of the grammar is important

- Many grammars generate the same language
- Tools are sensitive to the grammar.

Derivations

A *derivation* is a sequence of productions

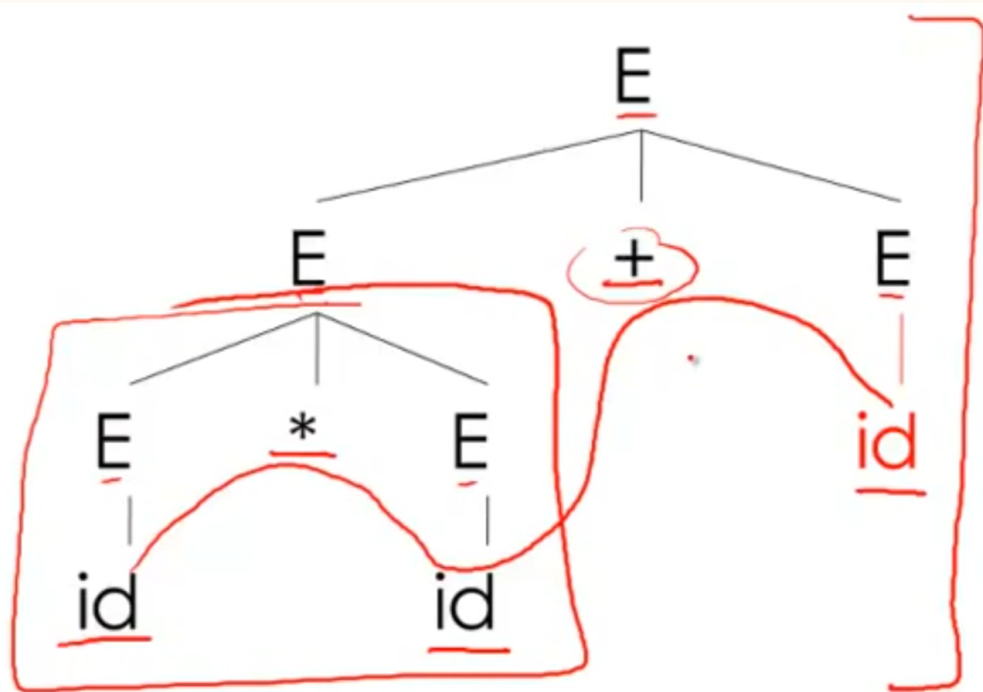
$$\underline{S} \rightarrow \underline{\dots} \rightarrow \underline{\dots} \rightarrow \underline{\dots} \rightarrow \underline{\dots} \rightarrow \underline{\dots}$$

A derivation can be drawn as a tree

- Start symbol is the tree's root
- For a production $\underline{X} \rightarrow Y_1 \dots Y_n$ add children $Y_1 \dots Y_n$ to node X



E
→ E + E
→ E * E + E
→ id * E + E
→ id * id + E
→ id * id + id



A parse tree has

- Terminals at leaves
- Non-terminals at the interior nodes.

An in-order traversal of the leaves is the original input.

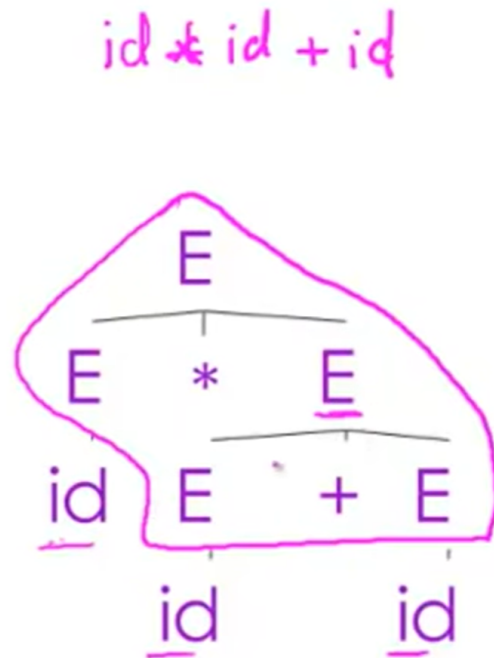
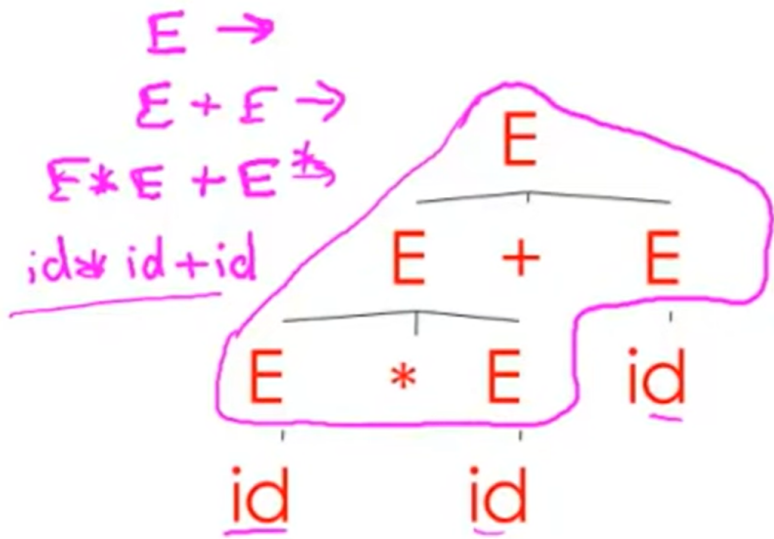
The parse tree shows the association of operations, the input string doesn't.

Note that right-most and left-most derivations have the same parse tree.

- We are not just interested in whether $s \in L(G)$
 - We need a parse tree for s
- A derivation defines a parse tree
 - But one parse tree may have many derivations
- Left-most and right-most derivations are important in parser implementation

Ambiguity

This string has two parse trees



$E \rightarrow$
 $E * E \rightarrow$
 $E * E + E \rightarrow$
 $id * id + id$

A grammar is ambiguous if it has more than one parse tree for some string.

- Equivalently, there is more than one right-most or left-most derivation for some string

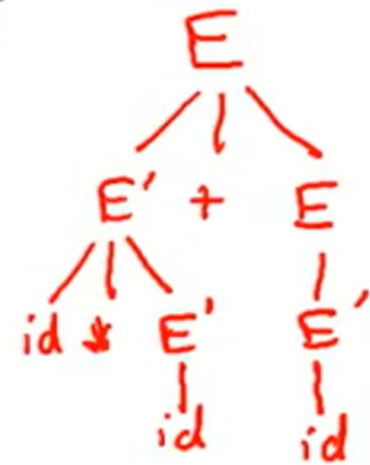
Ambiguity is BAD

- Leaves meaning of some programs ill-defined.

- There are several ways to handle ambiguity $id * id + id$

- Most direct method is to rewrite grammar unambiguously

$$\begin{cases} E \rightarrow E' + E \mid E' \\ E' \rightarrow id * E' \mid id \mid (E) * E' \mid (E) \end{cases}$$

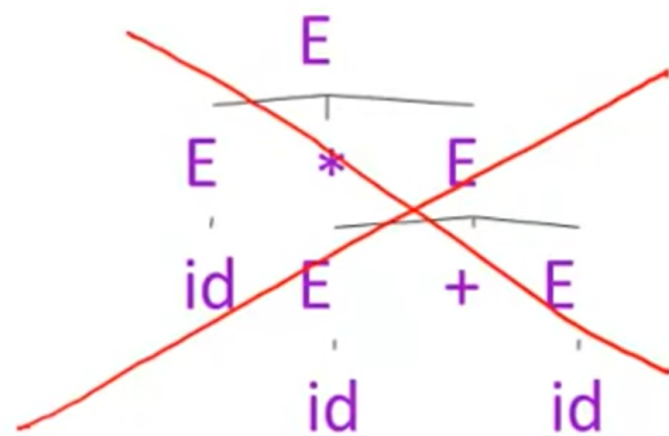
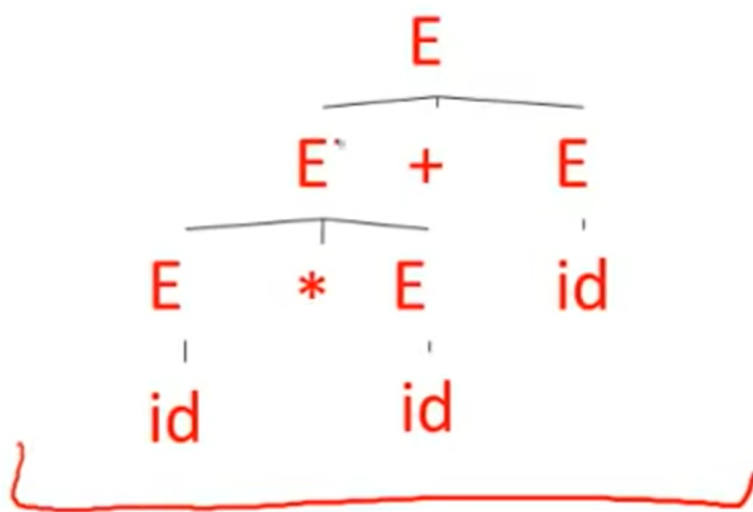


- Enforces precedence of $*$ over $+$

$$\begin{aligned} E &\rightarrow E' + E \rightarrow E' + E' + E \rightarrow E' + E' + E' + E \rightarrow \dots \rightarrow E' + \dots + E' \\ E' &\rightarrow id * E' \rightarrow id * id * E' \rightarrow id * id * id * E' \rightarrow \dots \rightarrow id * \dots * id \\ &\quad (E) \quad (E) \quad (E) \quad \dots \quad (E) \quad (E) \end{aligned}$$

$id * id + id$

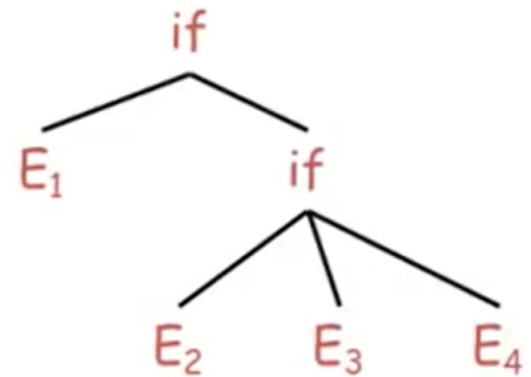
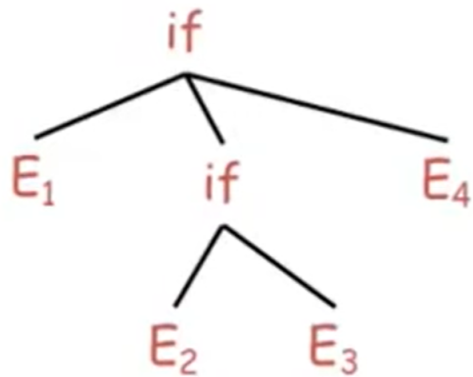
Ambiguity



- The expression

if E₁ then if E₂ then E₃ else E₄

has two parse trees



- else matches the closest unmatched then

$E \rightarrow \text{MIF}$

/* all then are matched */

| UIF

/* some then is unmatched */

$\text{MIF} \rightarrow \text{if } E \text{ then } \underline{\text{MIF}} \text{ else } \underline{\text{MIF}}$

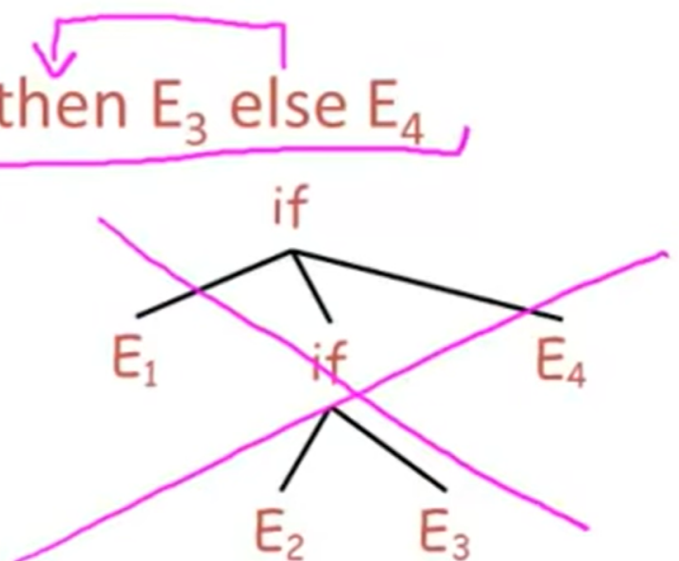
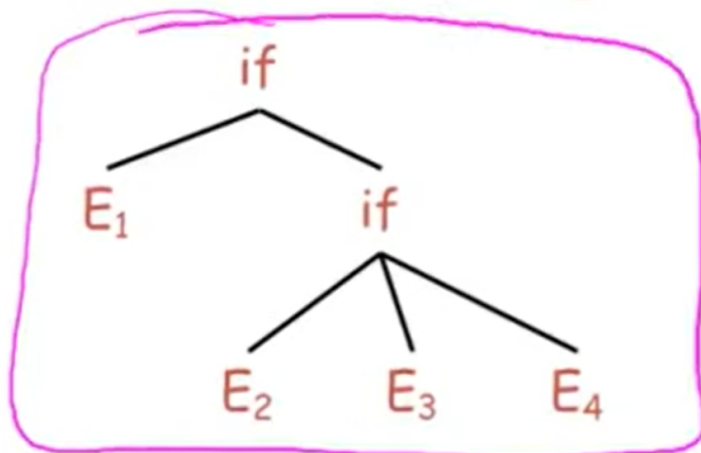
| OTHER

$\text{UIF} \rightarrow \text{if } E \text{ then } E$

| if E then MIF else UIF

UIF then

- The expression if E₁ then if E₂ then E₃ else E₄



Note: Impossible to convert automatically an ambiguous grammar to an unambiguous one.

Used with care, ambiguity can simplify the grammar.

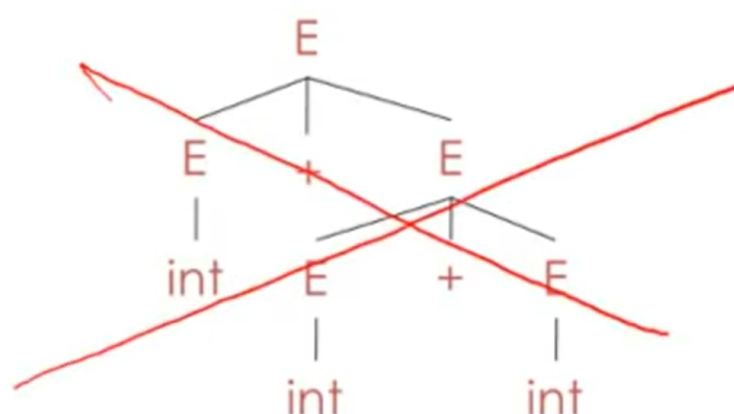
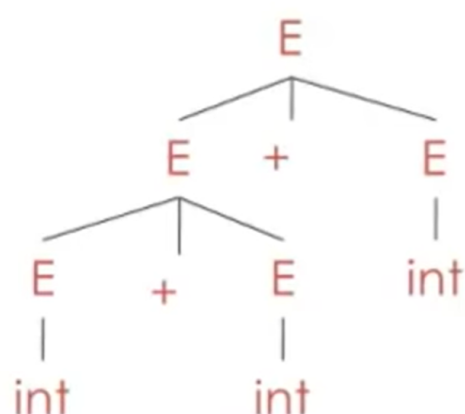
- Sometimes allows more natural definitions
- We need disambiguation mechanisms.

Instead of rewriting the grammar.

- Use the more natural (ambiguous) grammar
- Along with disambiguating declarations.

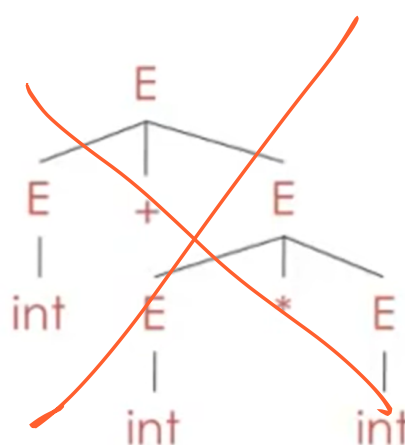
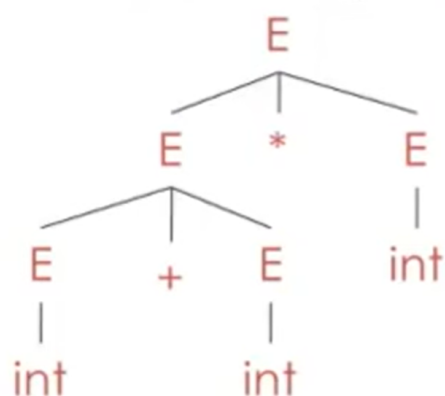
Most tools allow precedence and associativity declarations to disambiguate grammars.

- Consider the grammar $E \rightarrow E + E \mid \text{int}$
- Ambiguous: two parse trees of $\text{int} + \text{int} + \text{int}$



- Left associativity declaration: %left +
bison

- Consider the grammar $E \rightarrow E + E \mid E * E \mid \text{int}$
- And the string $\text{int} + \text{int} * \text{int}$



- Precedence declarations: $\left[\begin{array}{l} \underline{\%left +} \\ \underline{\%left *} \end{array} \right]$

Note: So the declaration rules out the parse tree on the left.

times takes precedence over plus.